AI for All - Tools

Yogesh Haribhau Kulkarni

YHK

OpenCode

Agenda

- What is OpenCode?
- Key Features & Capabilities
- Why OpenCode? The Case for It
- Architecture & Ecosystem
- Real-World Use Cases
- Live Demo Walkthroughs

YHK

What is OpenCode?

- Open-source AI coding agent built for the terminal (TUI).
- Brings full AI assistance directly to the command line, no IDE must.
- **IDE Integration possible**: Works with VS Code, Cursor, and any terminal-supporting IDE
- **GitHub Integration**: Automate issues and PRs directly from comments
- **Model-Agnostic:** Supports 75+ AI models (OpenAI, Claude, Groq, Ollama, and more).
  - *Local:* Ollama (fully offline, zero data egress)
  - *Cloud:* OpenAI, Anthropic Claude, Groq, Gemini, etc.
- **Privacy-Focused:** Runs locally or uses your own API keys, your code stays on your machine.
- **Built With:** Rust/Go for performance; Bubble Tea for the TUI layer.
- **Community Scale:** 1.5M+ monthly users · 80k+ GitHub stars (2026).

YHK

Capabilities

Think of it as having a senior developer sitting next to you, ready to help with:

- Writing and debugging code
- Understanding existing codebases
- Refactoring and optimization
- Answering technical questions
- Automating repetitive tasks

YHK

Key Features

- **Interactive TUI:** Smooth terminal UI built with Bubble Tea; keyboard-driven workflow.
- **Contextual Understanding:** Reads your entire repo, understands dependencies, structure, languages.
- **Agentic Workflows:** Plans changes, searches files, executes shell commands, modifies code autonomously.
- **Plan / Build Modes/Agents:** Review proposed changes before applying, deterministic control.
- **Slash Commands:** `/init`, `/help`, `/models`, `/share`, and more.
- **Context Mentions:** `@file:`, `@workspace` for scoped AI queries.

YHK

Core Integration

- **LSP Integration:** Language Server Protocol for real-time code intelligence (completions, diagnostics, go-to-def).
- **MCP (Model Context Protocol):** Plug in external tools and data sources to extend agent capabilities.
- **ACP Support:** Agent Communication Protocol for inter-agent coordination.
- **Agent Skills & Custom Tools:** Define reusable agent behaviors and wire in custom shell/API tools.
- **Rules & Permissions:** Fine-grained control over what the agent can read, write, or execute.
- **Multi-Provider Routing:** Failover and cost-based routing across model providers.

YHK

OpenCode vs Other Tools

Feature	OpenCode	Claude Code	Cursor
Open Source	✅	❌	❌
Free Tier	✅	Limited	Limited
Terminal-first	✅	✅	❌
Provider Agnostic	✅	❌ (Claude only)	❌
Local Models	✅	❌	✅
IDE Integration	✅	✅	Native

(Ref: OpenCode - nxcode.io

Installation

- **Quick Install (Recommended):**

  ```
  $ curl -fsSL https://opencode.ai/install | sh
  ```

- **Install npm/bun:** `npm i -g opencode-ai@latest` or `bun add -g opencode-ai`
- **Desktop App:** Download directly from opencode.ai/download for a native desktop experience.

YHK

Getting Started

- **First Step:** Run `/init` to ground the agent in your project.
- **Configure Provider**: `Global config ~/.config/opencode/opencode.json Or project-specific ./opencode.json`
- **Config Format:** TOML-based `opencode.toml`; supports project-level and global overrides.
- **Bring Your Own Keys:** Set `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, or point to Ollama.
- **Launch:** `opencode` in any project directory. Have conda env activated before.

YHK

Plan vs Build Mode

Switch between them using the Tab key.
Plan Mode:

- Read-only - Cannot make changes to your code
- Analyzes and explores your codebase
- Suggests implementation strategies
- Perfect for understanding before modifying

Build Mode (Default)

- Full access - Can read, write, and modify files
- Executes code changes
- Runs commands
- Best for actual development work

Pro Tip: Always start complex features in Plan mode to outline your approach, then switch to Build mode for implementation.

YHK

Essential Commands

- `/undo` Undo the last change
- `/redo` Redo an undone change
- `@` Fuzzy search for files in your project
- `Tab` Switch between Plan and Build mode
- `Cmd+Esc` Open OpenCode in IDE split view (Mac)
- `Ctrl+Esc` Open OpenCode in IDE split view (Windows/Linux)

YHK

AGENTS.md

For best results, create an AGENTS.md file in your project root. This helps OpenCode understand: Your project structure, Coding conventions, Preferred patterns, Technology stack.

```
## Project: My SaaS App

## Tech Stack
- Next.js 14 with App Router
- TypeScript
- Tailwind CSS
- PostgreSQL with Prisma

## Conventions
- Use functional components
- Prefer server components when possible
- Follow REST API naming conventions
- Write tests for all new features

## Structure
- /app - Next.js app router pages
- /components - Reusable UI components
- /lib - Utility functions and helpers
- /prisma - Database schema and migrations
```

Real-World Use Cases

- **Understanding a Codebase:**
  `You: Explain the authentication flow in this project` OpenCode will analyze your files and provide a detailed explanation of how authentication works.
- **Adding a Feature:** `You: Add a dark mode toggle to the settings page` OpenCode will: Find the settings page, Identify the theme system, Implement the toggle, Update necessary styles
- **Debugging:**
  `You: The login form is not submitting. Here's the error: [paste error]` OpenCode will trace the issue and suggest fixes.
- **Multi-File Refactoring:** Rename, restructure, or migrate patterns across the entire codebase atomically.
- **Documentation Generation:** Auto-generate README, API docs, and inline comments from code context.
- **Test Generation:** Scaffold unit/integration tests with awareness of existing test patterns.
- **Security Auditing:** Identify unsanitized inputs, SQL injection points, or insecure defaults.

YHK

Demos

Initialize & Understand

- **Goal:** Show how OpenCode scans and understands a new project from scratch.
- **Setup:** Open terminal inside any Git repository.
- **Launch:** `$ opencode`
- **Command:** `> /init`
- **What Happens:** OpenCode scans all local files, infers stack (languages, deps, structure), and writes `AGENTS.md`, the AI's instruction manual for this project.
- **Key Point:** Every subsequent interaction is grounded in this context, no hallucinated file paths.

YHK

Fix & Refactor

- **Goal:** Demonstrate AI-driven code modification on a real file.
- **Command:** `> @file:main.go fix the bug where user input isn't sanitized`
- **Agent Action:** Reads `main.go`, identifies the unsanitized input vector, proposes a targeted patch.
- **Review Flow:**
  - Press `Tab` → switch to **Plan** mode: inspect the proposed diff before anything touches disk.
  - Press `Tab` → switch to **Build** mode: apply the change.
- **Key Point:** You stay in control, the agent proposes, you decide.

YHK

Context-Aware Codebase Search

- **Goal:** Navigate a large, unfamiliar codebase using natural language.
- **Command:** `> @workspace where is the database connection configured?`
- **Agent Action:** Scans the entire workspace, returns exact file(s) and line(s) where DB config lives.
- **Key Point:** Replaces `grep -r`, `find`, and guesswork; understands *intent*, not just keywords.
- **Follow-up Capability:** Chain directly into a fix or refactor from the same session.

YHK

Add a Feature (Multi-File Editing)

- **Goal:** Show coordinated, multi-file changes from a single natural language prompt.
- **Command:**

```
> add a new API endpoint to delete user notes and update the README
```

- **Agent Plan:**
  - `routes/notes.go`, add `DELETE /notes/:id` route
  - `handlers/notes.go`, implement handler logic
  - `db/notes.go`, add delete query
  - `README.md`, document the new endpoint
- **Key Point:** Agent understands cross-file dependencies, no manual coordination needed.

YHK

Tips for Beginners

- Be Specific: Don't say "Make it better" but say, "Refactor the handleSubmit function to use async/await and add error handling"
- Provide Context: Don't say "Fix the bug" but say, "The login form throws 'undefined is not a function' when clicking submit. The error occurs in auth.ts line 45"
- Use Plan Mode First: For complex features, ask OpenCode to create a plan before implementation.
- Leverage Image Support: Drag and drop screenshots or mockups into the terminal—OpenCode can understand images!
- Create Project-Specific Configs: Different projects may need different settings. Use local opencode.json files.

YHK

Summary

Why Use OpenCode?

- **Privacy-First:** Code never leaves your machine by default; no telemetry, no vendor data retention.
- **Cost Efficiency:** No subscription fees, pay only for actual API token usage; local models are free.
- **No Vendor Lock-in:** Switch AI providers in seconds via config; one tool, all models.
- **Terminal-Native Speed:** Faster context-switch than IDE plugins; integrates directly into your existing shell workflow.
- **Transparent & Auditable:** Fully open-source (Apache 2.0), inspect, fork, or extend anything.
- **Enterprise-Ready:** Supports self-hosted LLMs and air-gapped (physically isolated from all other networks) environments.
- **Bottom Line:** Faster dev loop, zero lock-in, full code privacy, AI that works *with* your terminal.

YHK

Resources

- GitHub: `github.com/opencode-ai/opencode`
- Docs: `opencode.ai/docs`
- NXCode.io "OpenCode Tutorial 2026: Install & Setup After Anthropic Block (Updated Guide)"

YHK